

Démarche de réalisation et de programmation du premier prototype

Travail de maturité

10.05.2026

Abraxas

Isao Perregaux
isao@perregaux.net
Collège Calvin
301

1 TABLE DES MATIÈRES

2	Note	1
3	Matériel	3
	Commande 1	3
	Commande 2	4
4	Montage	4
5	Installation de l'OS	6
6	SSH par clés publiques	6
	Chiffrement asymétrique	7
	NAT-Loopback et Port mapping	7
	Changement de clé publique du Raspberry	8
7	Transfert de fichiers	9
	Dans la pratique	10
8	Paquets et librairies	10
9	Test des GPIO	11
10	Réglage de Serial Port	12
11	Usage de l'IMU	14
12	Usage du GPS	15
	Installation de Blinka	16
	PRÉCAUTION PARTICULIÈRE	17
	Installations de la bibliothèque relative au GPS	18
	Test du GPS	18
13	Usage de la caméra	21
14	Base de données SQL	22
15	Softwear	25
	Fichier de classes — classes.py	26
	Surveillance du bouton — interface.py	28
	Gestion de la session — master.py	30
	Gestion de l'IMU et du GPS — imu.py & gps.py	34
	imu.py	35
	gps.py	38
	Enregistrement de la vidéo — vid.py	41
	Finalisation de l'enregistrement de la vidéo — grp.py	45
16	Création des schémas	47

2 NOTE

Toutes les commandes à exécuter sur le Raspberry peuvent être exécutées immédiatement via SSH, qu'il soit interne ou externe au réseau, ou en NAT-Loopback. Ces commandes sont marquées avec `bash`, le nom du langage de la console Linux. Bien que les distributions Linux soient majoritairement similaires, il se peut que certaines commandes diffèrent. Celles-ci sont données pour Raspberry Pi OS, une déclinaison de Debian.

Les commandes à exécuter sur l'ordinateur distant sont marquées avec batch, le langage de l'invite de commandes Windows, cependant les commandes commençant par ssh ou scp sont universelles, elles fonctionnent aussi bien sur Linux, sur Windows que sur MacOS.

3 MATÉRIEL

Le choix du matériel a été une étape compliquée. En effet, je n'avais quasiment aucune connaissance en matière d'électronique, et choisir les composants demande de réfléchir à la compatibilité des protocoles, des tensions et des pinouts. De plus, les boutiques en lignes proposent des centaines de composants extrêmement similaires, mais cependant pas interchangeables. Faire la différence entre eux a parfois été un défi en soi. Tous les composants ne sont pas disponibles chez le même vendeur, ce qui m'a forcé à passer commande auprès de six entités différentes.

J'ai dû à plusieurs reprises recommander du matériel, parce que je n'avais pas pris le bon ou parce qu'il m'en manquait. Je m'attendais à ce que la principale difficulté soit dans la partie softwear, mais le choix du hardware s'est avéré particulièrement chronophage.

Commande 1

La première commande comprend le matériel de base, tel que le Raspberry lui-même, l'IMU, le GPS et la caméra. L'anémomètre ne sera commandé qu'ultérieurement.

Type de composant	Lien	Nom du composant
IMU	lien	ICM20948, Breakout Pimoroni
Micro-ordinateur	lien	Raspberry Pi Zero 2 WH
GPS	lien	MTK 3339, Adafruit Ultimate GPS Breakout
Caméra	lien	Original Raspberry Pi Kamera Module V2
Power boost	lien	Adafruit PowerBoost 1000C
Batterie	lien	LiPO 8000 mAh 2C



Fig. 1. – Photo du matériel de la première commande avant montage.

Commande 2

En passant la première commande, j'ignorais qu'il existait deux normes de câble CSI. Le câble fourni avec la caméra est un CSI 15 contacts, mais l'entrée sur le Raspberry est un mini-CSI 22 contacts. Il a donc fallu racheter une nappe pour raccorder les deux composants. On voit le détail de cette nappe sur la Fig. 8.

Type de composant	Lien	Nom du composant
Nappe CSI	lien	CSI (15 contacts) ↔ Mini-CSI (22 contacts)

4 MONTAGE

Tous les schémas des montages ont été créés à l'aide du logiciel Fritzing.

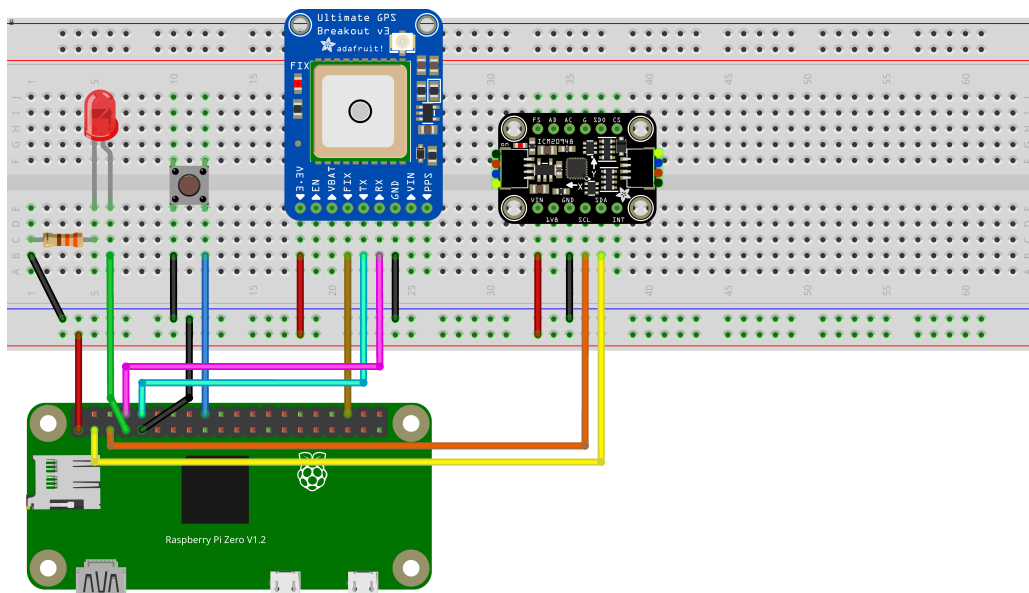


Fig. 2. – Représentation du montage effectué.

Sur cette représentation, on a utilisé pour l'IMU un breakout Adafruit de l'ICM20948, alors que le breakout réellement utilisé est un Pimoroni. Cependant, la puce utilisée et leur pinout est identique, la seule différence est donc visuelle. Ce choix est dû au fait que Pimoroni n'a pas créé de représentation graphique de son breakout utilisable dans Fritzing.

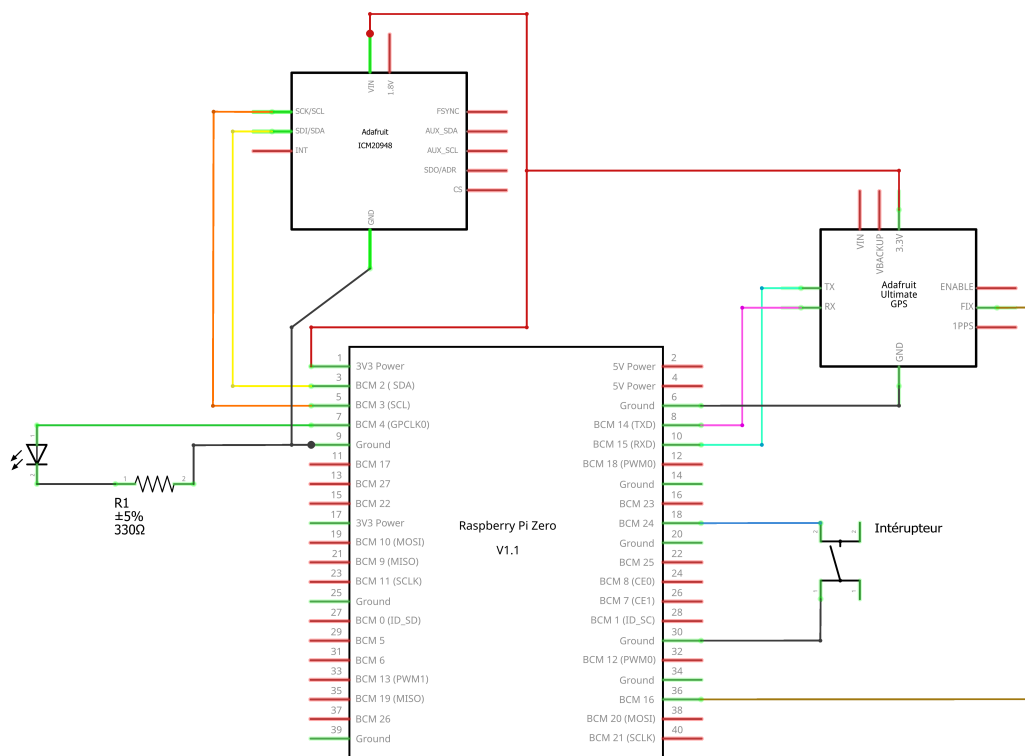


Fig. 3. – Représentation schématique du montage effectué.

Pour procéder au montage, il a fallu souder chaque broche de l'IMU et du GPS, soit 14. Cet exercice de minutie a représenté une difficulté majeure, car les broches sont petites et extrêmement rapprochées les unes des autres (pas de 2.54mm), or, les soudures ne doivent en aucun cas se toucher. De plus, le fer à souder ne doit pas rester proche du composant trop longtemps, sous peine de faire fondre le plastique du support.

5 INSTALLATION DE L'OS

L'OS installée est *Raspberry Pi OS Lite (64-bit)*. Dépourvue d'interface graphique, elle a été choisie en raison du faible espace disque utilisé, de sa simplicité d'utilisation et d'installation, de la grande documentation la concernant, mais surtout pour la faible charge processeur engendrée. En effet, la gestion des ressources, limitées sur un *Raspberry Pi Zero 2WH* est un défi majeur de ce projet.

Pour son installation, le logiciel *Raspberry Pi Imager* a été utilisé. On télécharge via ce logiciel l'OS sur son ordinateur, avant de la flasher sur la carte SD qui servira de disque au Raspberry. Lors de l'installation, on configure le compte utilisateur, le mot de passe utilisateur, le mot de passe *root*, le Wi-Fi (SSID et mot de passe), ainsi que l'usage du *SSH* avec authentification par chiffrement asymétrique (détaillé au Chapitre 6). Ainsi, une fois la carte SD insérée dans le Raspberry, tout est immédiatement fonctionnel, il n'est même pas nécessaire de le brancher à un écran ou à un clavier pour l'utiliser.

6 SSH PAR CLÉS PUBLIQUES

SSH est l'abréviation de *Secure Shell*. Ce protocole permet d'accéder à la console du Raspberry depuis un autre ordinateur avec la syntaxe suivante :

```
batch
ssh user@ip_raspberry
```

Avec *user* le nom d'utilisateur et *ip_raspberry* l'adresse ip du Raspberry (Ici 192.168.1.13). Par défaut, quand une demande de connexion entrante est détectée, le Raspberry demande le mot de passe du compte *user* pour autoriser l'accès à la console. Cependant cette méthode d'authentification n'est pas la plus sécurisée, car un mot de passe peut être cassé par force brute. Pour cette raison, il a été choisi d'utiliser le chiffrement asymétrique.

Si l'on choisit d'utiliser *SSH* pour l'accès à distance au lieu de systèmes de plus haut niveau plus instinctifs, c'est pour son faible usage de ressources, sa large compatibilité, sa sécurité optimale et sa stabilité éprouvée, mais justement car c'est un processus de bas niveau. En effet, ces processus sont bien plus permissifs et configurables dans le

cas de projet de développement, et ont l'avantage de ne nécessiter aucun téléchargement supplémentaire.

Chiffrement asymétrique

Cette méthode peut être activée après l'installation de l'OS, mais pour des raisons de simplicité, elle a été configurée directement avec *Raspberry Pi Imager*. Pour se connecter en *SSH* de cette manière, il est nécessaire d'avoir une clé publique et une clé privée sur chaque appareil. La génération de ces clés se fait en utilisant

```
batch  
ssh-keygen
```

sur l'ordinateur depuis lequel on veut accéder au Raspberry. Cette commande fonctionne aussi bien sur Linux, sur MacOS que sur Windows. Dans les trois cas, on suit les indications données par la console / le terminal sans renseigner plus d'options pour garder la configuration par défaut. Au terme de cette opération, on obtient une clé publique et une clé privée, stockées respectivement dans `.ssh/id_rsa.pub` et dans `.ssh/id_rsa`. Lors de la configuration de l'OS avec *Raspberry Pi Imager*, on renseigne la clé publique. La création de clés sur le Raspberry se fait automatiquement. Ainsi, dès le premier *boot* du Raspberry, on peut y accéder de façon distante sans renseigner de mot de passe, à condition d'être connecté au même réseau.

NAT-Loopback et Port mapping

Afin d'accéder au Raspberry depuis l'extérieur du réseau auquel il est connecté, on doit réaliser une opération sur la box (souvent 192.168.1.1). Le port par défaut du SSH est le 22. Par conséquent, l'objectif est d'ouvrir le port 22 de la box et de rediriger les informations arrivant dessus vers le Raspberry. La marche à suivre dépend de la box, mais il est généralement possible d'y accéder depuis un navigateur en tapant 192.168.1.1 dans la barre de recherche.

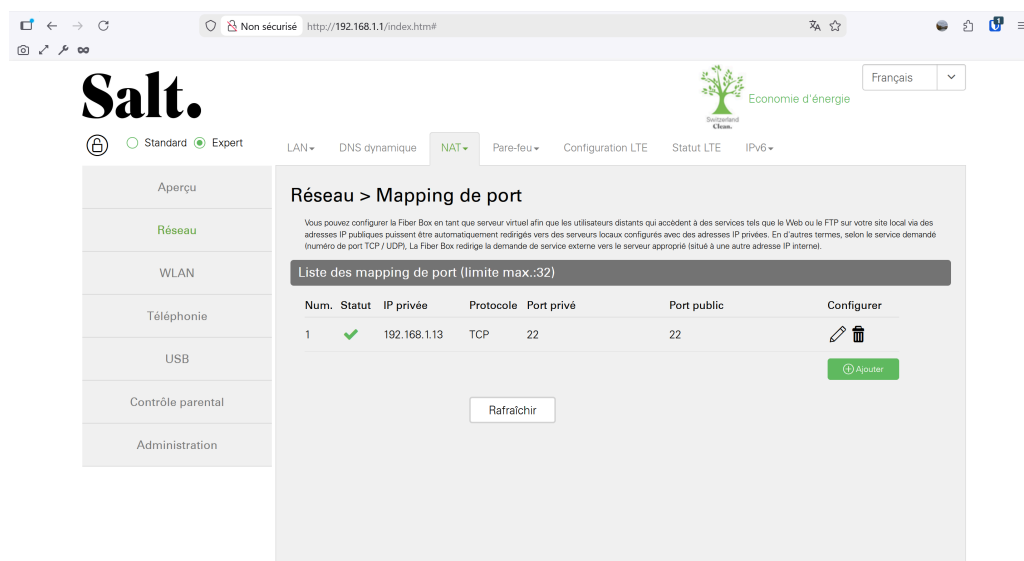


Fig. 4. – Page de configuration de mapping de port sur une box Salt.

Pour accéder au Raspberry depuis l'extérieur une fois le mapping de port réglé, il suffit d'exécuter

```
batch  
ssh user@ip_publicue_box
```

L'`ip_publicue_box` est l'adresse IP du réseau visible depuis l'extérieur. On peut la trouver facilement grâce à des sites tels que <https://www.myip.com/>.

Ensuite, il faut autoriser le NAT-Loopback afin de permettre à l'ordinateur de se connecter au Raspberry depuis le même réseau, alors même que la porte d'entrée privilégiée est tournée vers l'extérieur. Le NAT-Loopback permet à la box d'interpréter des requêtes venant du réseau qu'elles génèrent comme des requêtes venant de l'extérieur. C'est à dire qu'une demande de connexion adressée à son IP publique faite depuis l'intérieur du réseau ne pourra pas être interprétée correctement sans NAT-Loopback.

Changement de clé publique du Raspberry

Il arrive parfois que cette clé change automatiquement. La nouvelle clé différera donc de celle enregistrée dans `.ssh/known_hosts`. Cette différence lève l'inquiétante erreur suivante :

```
bash
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@      WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!      @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the rsa key sent by the remote host is
SHA256:7QeljpfXjWS1Q6INCVhjp1Hwn+MIA23Tc0cgTKcIgVt4.
Please contact your system administrator.
Add correct host key in C:\\Users\\isao\\.ssh\\known_hosts to get rid of this message.
Offending ECDSA key in C:\\Users\\isao\\.ssh\\known_hosts:77
Host key for 192.168.1.13 has changed and you have requested strict checking.
Host key verification failed.
```

Pour se débarrasser de cette erreur, il convient d'exécuter :

```
batch
ssh-keygen -R 192.168.1.13
```

Cette commande supprime toutes les clés publiques de `.ssh/known_hosts` relatives à cette adresse IP. Cela permet une nouvelle connexion qu'il faudra valider en tapant y quand on y est appelé. ATTENTION ! N'exécuter cette commande seulement si l'on est certain que personne n'est en train de conduire un MITM, ce serait lui ouvrir la porte du réseau.

7 TRANSFERT DE FICHIERS

Au cours de cette démarche, il sera à plusieurs reprises nécessaire de transférer du code python sur le Raspberry. Pour ce faire, il existe plusieurs méthodes, nous allons en présenter deux :

- Avec SCP :

On enregistre sur l'ordinateur distant le fichier python. On le copie ensuite sur le Raspberry depuis l'ordinateur distant en utilisant SCP, un protocole de SSH, avec la syntaxe suivante :

```
batch
scp chemin\\absolu\\de\\départ\\script.py user@192.168.1.13:/home/user/destination/
script.py
```

Attention à bien travailler en chemin absolu et non en chemin relatif. Sur Windows, les « / » dans les chemins d'accès sont retournés.

- Avec nano :

Directement sur le Raspberry, on exécute :

```
bash
cd /home/user/destination
nano script.py
```

Puis on colle le code dans l'éditeur de texte qui s'ouvre. Un tel copié-collé n'est possible que si l'on est connecté via SSH au Raspberry, si ce n'est pas le cas, mieux vaut utiliser SCP. Pour enregistrer, on fait Ctrl+X, puis y pour valider, et Enter pour sortir de nano.

Dans la pratique

Dans le cadre de ce TM, j'ai créé un script batch utilisant SCP pour directement transférer tous les fichiers du softwear (présentés dans le Chapitre 15). Ainsi, à chaque modification, il me suffisait de l'exécuter pour actualiser automatiquement chaque fichier sur mon Raspberry.

```
batch
scp -r "C:\Users\isao\Documents\GitHub\Abraxas\*.py"
user@51.154.XXX.XXX:/home/user
```

8 PAQUETS ET LIBRAIRIES

On commence par faire les mises-à-jour standards et par installer tree, pip et python3-full :

```
bash
sudo apt-get update
sudo apt-get -y upgrade
sudo apt install tree
sudo apt install python3-pip
sudo apt install python3-full
```

Toutes les autres paquets qui pourraient être nécessaires ultérieurement pourront simplement être installés de la même manière.

Pour installer des librairies python, on utilise pip. On doit par exemple installer time :

```
bash
pip3 install time
```

Dans le cas où pip renverrait l'erreur suivante :

```
bash
error: externally-managed-environment

× This environment is externally managed
└> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have python3-full installed.

    For more information visit http://rptl.io/venv

note: If you believe this is a mistake, please contact your Python installation
or OS distribution provider. You can override this, at the risk of breaking your
Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
```

On peut ajouter une option pour forcer l'installation à la fin de la ligne :

```
bash
pip3 install time --break-system-packages
```

9 TEST DES GPIO

Pour réaliser un premier essai, on souhaite faire clignoter une LED connectée au GPIO 7, soit le BCM 4. On crée le fichier `blink.py` avec le code suivant sur le Raspberry (par exemple dans `/home/user`) :


```
python
import RPi.GPIO as GPIO
import time

LED_PIN = 4 # GPIO 7 = BCM 4

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)

try:
    while True:
        GPIO.output(LED_PIN, GPIO.HIGH)
        time.sleep(0.5)
        GPIO.output(LED_PIN, GPIO.LOW)
        time.sleep(0.5)

except KeyboardInterrupt:
    pass

finally:
    GPIO.cleanup()
```

On installe ensuite la librairie RPi.GPIO sur le Raspberry :

```
bash
pip3 install RPi.GPIO
```

On lance finalement le script sur le Raspberry avec :

```
bash
python3 blink.py
```

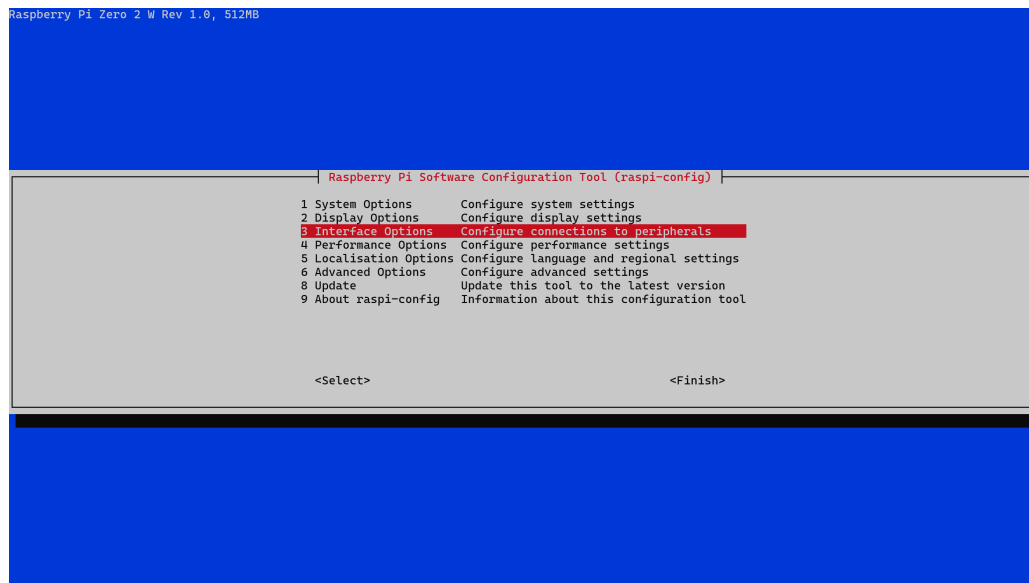
On observe la LED clignoter avec une période d'une demie-seconde, on peut l'arrêter avec un KeyboardInterrupt (Ctrl+C).

10 RÉGLAGE DE SERIAL PORT

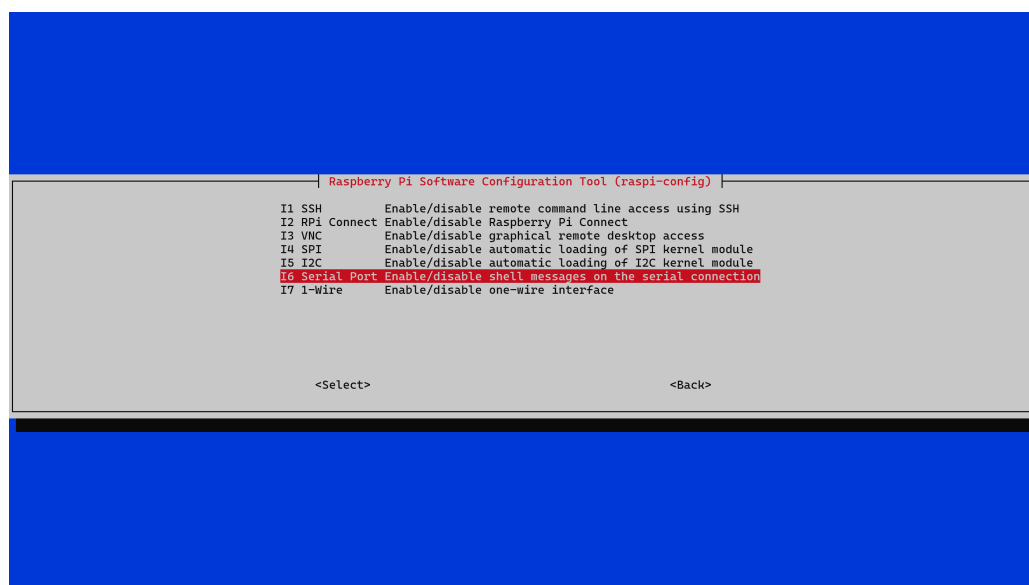
Pour permettre au port UART (TxD et RxD) de fonctionner correctement, on doit procéder à un réglage du port série : on veut désactiver le serial login shell et activer le serial interface. On exécute :

```
bash
sudo raspi-config
```

Ensuite, on sélectionne Interface Option



Puis Serial Port



On choisit no pour le serial login shell, yes pour le serial interface. On devrait avoir une fenêtre de ce type :



On valide, puis on entre finish, et afin que les changements prennent effet (si le reboot n'est pas déjà proposé à la sortie de raspi-config) :

```
bash
sudo reboot
```

Le Raspberry va redémarrer, la connexion SSH va se clore, mais il sera possible de se reconnecter après quelques minutes.

11 USAGE DE L'IMU

L'IMU 9DoF fonctionne en bus I²C, soit avec les ports SDA (informations) et SCL (horloge). Pour l'initialiser, on installe les bibliothèques, les paquets et les scripts du repository Github¹ créé par Pimoroni. Ensuite, on exécute le script d'initialisation `install.sh`. Pour procéder ainsi, on a besoin d'avoir `git` installé :

```
bash
sudo apt install git
```

On exécute ensuite les commandes suivantes :

```
bash
cd /home/user
git clone https://github.com/pimoroni/icm20948-python
cd /home/user/icm20948-python
./install.sh
```

¹Accessibles via <https://github.com/pimoroni/icm20948-python>

Au cours de l'installation, il nous est demandé si l'on souhaite télécharger des exemple. Cela peut-être utile pour tester le bon fonctionnement de l'IMU. On valide avec `y`. On refuse par contre la documentation avec `n`, on ne veut pas en encombrer le Raspberry, et elle est accessible sur le repository. Il nous reste encore à installer la librairie `icm20948` depuis PyPi :

```
bash
pip3 install icm20948
```

On teste l'IMU en exécutant le script d'exemple `read-all.sh` :

```
bash
python3 /home/user/icm20948-python/examples/read-all.py
```

On devrait voir défiler des blocs d'information de ce type :

```
python
Accel: -0.16 00.00 01.04
Gyro:  -1.00 01.84 -0.75
Mag:   29.85 -41.25 128.85
```

Pour l'arrêter, on fait un `KeyboardInterrupt` (Ctrl+C).

12 USAGE DU GPS

Avant de commencer l'initialisation du GPS, s'assurer que le réglage de Serial Port (Chapitre 10) a bien été effectué. Le GPS fonctionne en UART asynchrone, c'est à dire avec les ports `RxD` et `TxD`. Contrairement au bus `I2C`, il ne s'agit pas d'une architecture maître/esclave, mais d'un processus point to point, permettant des interactions plus complexes entre le Raspberry et le GPS.

Pour l'initialisation du GPS, on s'est appuyé sur les guide publiés par Adafruit disponibles via les liens suivants :

- [Mode d'emploi du GPS \(PDF\)](#)
- [Exécution du code Circuit Python sans CircuitPython](#)
- [Installer Blinka sur un Raspberry Pi](#)

Une partie des bibliothèques, des paquets et des scripts de test viennent de ce repository :

- [Repository Github Adafruit_CircuitPython_GPS](#)

La démarche exacte a dû être adaptée aux besoins spécifiques du projet. On la retranscrit donc entièrement ici.

Le *Adafruit Ultimate GPS Breakout* fonctionne selon une logique CircuitPython, une version allégée de Python. Pour pouvoir le faire tourner sur un Raspberry (nativement dépourvu de CircuitPython), il faut installer un interpréteur : Blinka.

Installation de Blinka

On commence par upgrader les `setuptools` :

```
bash
sudo apt install --upgrade python3-setuptools
```

À ce moment là, le Raspberry demandera peut-être un reboot. On l'effectue si nécessaire, puis on installe un environnement virtuel pour y faire tourner Blinka.

```
bash
cd
sudo apt install python3-venv
python3 -m venv env --system-site-packages
```

Puis on l'active :

```
bash
source env/bin/activate
```

On procède ensuite à l'installation automatique de Blinka à l'aide d'un script fourni par Adafruit :

```
bash
cd
pip3 install --upgrade adafruit-python-shell
wget https://raw.githubusercontent.com/adafruit/Raspberry-Pi-Installer-Scripts/
master/raspi-blinka.py
sudo -E env PATH=$PATH python3 raspi-blinka.py
```

Le processus va demander une validation, on y procède en cliquant `y`. L'installation prends quelques minutes, au terme desquelles le Raspberry demande un reboot. On l'accepte en cliquant encore `y`.

On crée le fichier `blinkatest.py` (par exemple dans `/home/user`) avec le code suivant :

```
python
import board
import digitalio
import busio

print("Hello, blinka!")

# Try to create a Digital input
pin = digitalio.DigitalInOut(board.D4)
print("Digital IO ok!")

# Try to create an I2C device
i2c = busio.I2C(board.SCL, board.SDA)
print("I2C ok!")

# Try to create an SPI device
spi = busio.SPI(board.SCLK, board.MOSI, board.MISO)
print("SPI ok!")

print("done!")
```

On lance ensuite le script python pour tester Blinka :

```
bash
source env/bin/activate
python3 blinkatest.py
```

On devrait avoir :

```
bash
Hello, blinka!
Digital IO ok!
I2C ok!
SPI ok!
done!
```

PRÉCAUTION PARTICULIÈRE

CircuitPython contient une librairie nommée board, or Python possède une librairie du même nom. Tenter d'installer la librairie Python sur tout le système reviendrait à casser Blinka de façon irrémédiable, et il faudrait flasher à nouveau une OS vierge sur la carte SD pour recommencer toute l'installation (Expérience vécue). Si l'erreur suivante est levée :

```
bash
Traceback (most recent call last):
  File "/home/user/blinkatest.py", line 1, in <module>
    import board
ModuleNotFoundError: No module named 'board'
```

Ne surtout pas tenter de la résoudre en important board. Le problème n'est pas que cette bibliothèque n'est pas importée : elle bien présente, mais uniquement en CircuitPython dans le venv de Blinka. Pour se débarrasser de cette erreur, il faut vérifier que l'on se trouve bien dans l'environnement virtuel voulu, et au besoin s'y rendre à nouveau avec

```
bash
source env/bin/activate
```

Si on a ultérieurement besoin de board de Python (bien qu'il serait préférable de trouver un autre moyen), il faut veiller à circonscrire son installation à un environnement virtuel séparé de Blinka.

Installations de la bibliothèque relative au GPS

On doit d'abord installer adafruit-circuitpython-gps (des DEPRECATION pourraient être levées, mais elles sont sans gravité) :

```
bash
sudo pip3 install adafruit-circuitpython-gps --break-system-packages
--root-user-action=ignore
```

L'option `--root-user-action` avec l'attribut `ignore` permet d'éviter l'erreur suivante quand on exécute pip en sudo :

```
bash
WARNING: Running pip as the 'root' user can result in broken permissions and
conflicting behaviour with the system package manager, possibly rendering your
system unusable. It is recommended to use a virtual environment instead:
https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know
what you are doing and want to suppress this warning.
```

Test du GPS

On crée le fichier `GPS-test.py` (par exemple dans `/home/user`) avec le code suivant² :

²Ce code est une version modifiée d'un code d'exemple sous licence appartenant à Adafruit. Les seules modifications qui y ont été apporté sont la suppression de certains commentaires et l'adaptation pour une connexion via serial au lieu de busio. Le code original est disponible à https://github.com/adafruit/Adafruit_CircuitPython_GPS/blob/main/examples/gps_simpletest.py

```
python
# SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
# SPDX-License-Identifier: MIT
# Will wait for a fix and print a message every second with the current location
and other details.
import time
import board
import busio
import adafruit_gps
import serial

# Create a serial connection for the GPS connection using default speed
rx = board.RX
tx = board.TX
uart = serial.Serial("/dev/serial0", baudrate=9600, timeout=10)

# Create a GPS module instance.
gps = adafruit_gps.GPS(uart, debug=False)

# Turn on the basic GGA and RMC info
gps.send_command(b"PMTK314,0,1,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0")

# Set update rate to once a second (1hz)
gps.send_command(b"PMTK220,1000")

# Main loop runs forever printing the location, etc. every second.
last_print = time.monotonic()
while True:
    gps.update()
    # Every second print out current location details if there's a fix.
    current = time.monotonic()
    if current - last_print >= 1.0:
        last_print = current
        if not gps.has_fix:
            # Try again if we don't have a fix yet.
            print("Waiting for fix...")
            continue
        # If we have a fix (gps.has_fix is true)
        print("=" * 40)
        print(
            "Fix timestamp: {}/{}/{} {:02}:{:02}:{:02}".format( # noqa: UP032
                gps.timestamp_utc.tm_mon,
                gps.timestamp_utc.tm_mday,
                gps.timestamp_utc.tm_year,
                gps.timestamp_utc.tm_hour,
                gps.timestamp_utc.tm_min,
                gps.timestamp_utc.tm_sec,
            )
        )
    )
```



```
python
# gps.send_command(b'PMTK220,500')
print(f"Latitude: {gps.latitude:.6f} degrees")
    print(f"Longitude: {gps.longitude:.6f} degrees")
        print(f"Precise Latitude: {gps.latitude_degrees} degs,
{gps.latitude_minutes:2.4f} mins")
            print(f"Precise Longitude: {gps.longitude_degrees} degs,
{gps.longitude_minutes:2.4f} mins")
    print(f"Fix quality: {gps.fix_quality}")
    if gps.satellites is not None:
        print(f"# satellites: {gps.satellites}")
    if gps.altitude_m is not None:
        print(f"Altitude: {gps.altitude_m} meters")
    if gps.speed_knots is not None:
        print(f"Speed: {gps.speed_knots} knots")
    if gps.speed_kmh is not None:
        print(f"Speed: {gps.speed_kmh} km/h")
    if gps.track_angle_deg is not None:
        print(f"Track angle: {gps.track_angle_deg} degrees")
    if gps.horizontal_dilution is not None:
        print(f"Horizontal dilution: {gps.horizontal_dilution}")
    if gps.height_geoid is not None:
        print(f"Height geoid: {gps.height_geoid} meters")
```

Pour le lancer :

```
bash
python3 GPS-test.py
```

Pour que le GPS acquière un premier fix, il peut être nécessaire de le laisser pendant plusieurs minutes (voire heures) à proximité d'une fenêtre ou à l'extérieure, antenne pointée vers le ciel. Le temps nécessaire varie en fonction de l'environnement physique, de la météo, et de la présence de facteur perturbateurs tels qu'une grande densité d'onde. Une fois le GPS connecté, on devrait voir défiler des blocs d'information de ce type :

```
python
Fix timestamp: 2/9/2026 20:26:21
Latitude: 46.188553 degrees
Longitude: 6.145146 degrees
Precise Latitude: 46 degs, 11.3132 mins
Precise Longitude: 6 degs, 8.7088 mins
Fix quality: 1
# satellites: 4
Altitude: 107.4 meters
Speed: 0.93 knots
Speed: 1.79 km/h
Track angle: 355.59 degrees
Horizontal dilution: 3.43
Height geoid: 48.2 meters
```

Ce code a été adapté depuis une version publiée par Adafruit sur Github³ qui était originellement prévue pour des microcontrôleurs avec CircuitPython nativement installé. Le fait que le code soit interprété par Blinka a nécessité quelques ajustements pour créer une correspondance au niveau du pinout.

Ces ajustements ont été une difficulté importante, car la question des environnements virtuels, de la différence entre CircuitPython et Python, des protocoles de communication et des ports physiques sont très nouvelles pour moi. Or, ces modifications regroupaient toutes ces problématiques. De plus, si Blinka et CircuitPython font l'objet d'une documentation de qualité, la question de la correspondance au niveau physique reste plus floue, et les informations disponibles, de sources amatrices, sont souvent peu claires, obsolètes, ou simplement fausses.

13 USAGE DE LA CAMÉRA

L'initialisation de la caméra est une étape que j'avais mal anticipée. Pour la connecter au Raspberry, j'avais prévu un câble CSI classique, mais il s'est avéré que la nappe de connexion CSI du *Raspberry Pi Zero 2* est particulière. Elle ne possède pas 15 contacts (comme la majorité des CSI), mais 22. Il a donc fallu se procurer un nouveau câble spécifique. En ayant poussé mes recherches au moment du prototypage initial, j'aurais pu éviter cela.

Une fois le matériel approprié à disposition, l'installation physique est facile. Aucune soudure n'est nécessaire. Il suffit d'ouvrir les ports CSI de la caméra et du Raspberry en tirant délicatement sur le loquet noir, puis d'insérer la nappe dans le port. Les contacts de la nappe doivent faire face à ceux du port. On referme ensuite les loquets, et le câble est verrouillé.

³disponible sur
https://github.com/adafruit/Adafruit_CircuitPython_GPS/blob/main/examples/gps_simpletest.py

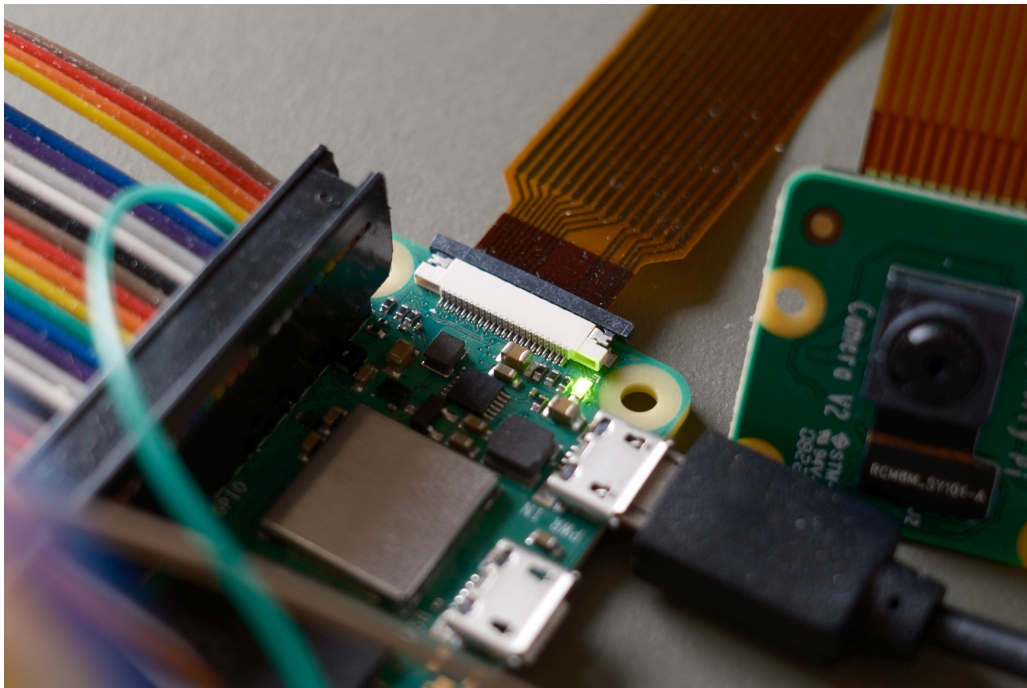


Fig. 8. – La nappe CSI dorée entre dans le port sur le Raspberry

Une fois le raccordement effectué, aucun ajustement logiciel supplémentaire n'est nécessaire. On peut directement utiliser la caméra. On la teste avec :

```
bash
rpivid --help
```

Après un délai de cinq seconde dans lequel l'image serait affichée si on avait une interface graphique, l'image est capturée et enregistrée dans le répertoire courant en tant que `test.jpg`. On la supprime pour ne pas encombrer le stockage.

```
bash
rm test.jpg
```

14 BASE DE DONNÉES SQL

Pour l'enregistrement de données de l'IMU et du GPS, on choisit une base de donnée SQLite. Elle permet de sauver rapidement et simplement les informations depuis python. De plus, une telle base de donnée a l'avantage d'être très robuste tout en conservant une organisation entre les données (ce que ne ferait pas un json par exemple).

On définit trois tables : `sessions`, `imu_data` et `gps_data`. Chaque table possède une clé primaire `id` en plus des autres champs. De plus, les tables `imu_data` et `gps_data` ont une clé étrangère permettant de relier chacun de leur enregistrement à une session (enregist-

trement de la table sessions). Ainsi, il sera facile de séparer les mesures des différents entraînements.

gps_data et imu_data sont séparées, car leurs enregistrements sont asynchrones. En effet, on ne peut pas enregistrer les données de l'IMU et du GPS dans une même table si les informations que l'on reçoit d'eux n'arrivent pas en même temps.

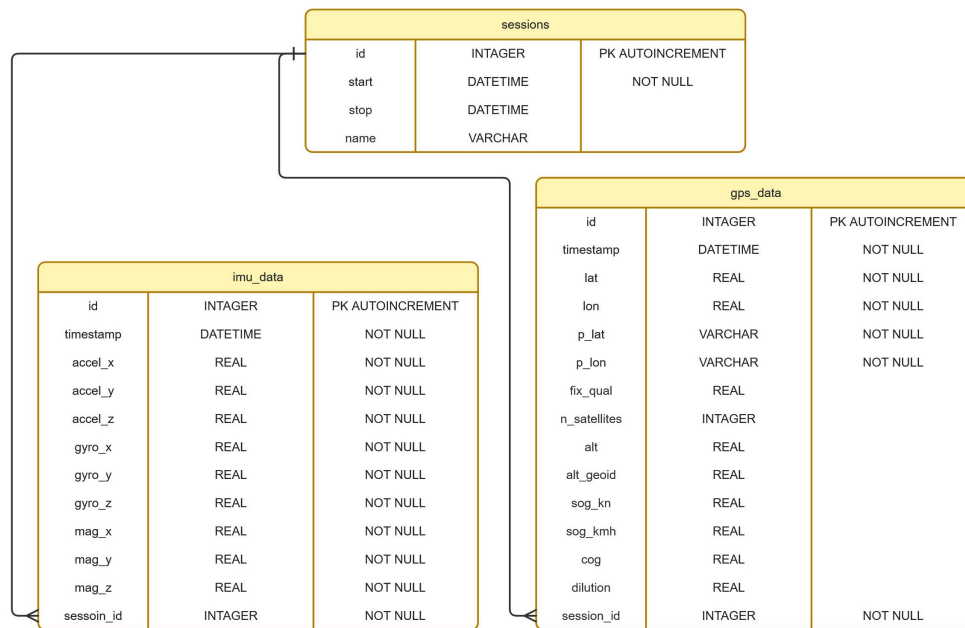


Fig. 9. – Schéma de la base de donnée relationnelle sys.db

Avant même de commencer à vouloir travailler avec une base de donnée en local, il faut installer sqlite.

```
bash
sudo apt install sqlite3
```

Pour construire la base de donnée dont le schéma est présenté ci-dessus, on commence par créer le fichier sys.db. On l'ouvre ensuite avec sqlite.

```
bash
touch sys.db
sqlite3 sys.db
```

On construit ensuite la première table en envoyant la requête SQL suivante :

```
SQL
CREATE TABLE sessions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,

  start DATETIME NOT NULL,
  stop DATETIME,

  name VARCHAR);
```

Pour chaque champ, on a spécifié le nom, le type et un éventuel contrainte. On fait de même pour les autres tables.

```
SQL
CREATE TABLE imu_data (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  timestamp DATETIME NOT NULL,

  accel_x REAL NOT NULL,
  accel_y REAL NOT NULL,
  accel_z REAL NOT NULL,

  gyro_x REAL NOT NULL,
  gyro_y REAL NOT NULL,
  gyro_z REAL NOT NULL,

  mag_x REAL NOT NULL,
  mag_y REAL NOT NULL,
  mag_z REAL NOT NULL,

  session_id INTAGER NOT NULL);
```

Puis :

```
SQL
CREATE TABLE gps_data (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  timestamp DATETIME NOT NULL,

  lat REAL NOT NULL,
  lon REAL NOT NULL,
  p_lat VARCHAR NOT NULL,
  p_lon VARCHAR NOT NULL,

  fix_qual REAL,
  n_satellites INTAGER,

  alt REAL,
  alt_geoid REAL,

  sog_kn REAL NOT NULL,
  sog_kmh REAL,
  cog REAL NOT NULL,

  dilution REAL,
  session_id INTAGER NOT NULL);
```

On quitte finalement la base de donnée (l'enregistrement est automatique).

```
SQL
.exit
```

Pour consulter le contenu d'une table (p.ex imu_data) de façon visuelle, on peut exécuter :

```
bash
sqlite3 sys.db -header -column "SELECT * FROM imu_data;"
```

15 SOFTWARE

Le fonctionnement du prototype est le suivant : on appuie sur un bouton pour lancer une session. Les enregistrements de l'IMU, du GPS et de la caméra commencent. On appuie une deuxième fois sur le même bouton pour arrêter la session. Les enregistrements se finalisent et le prototype reste en stand-by, prêt pour une nouvelle session.

Pour ce faire, on procède comme suit : un code python `interface.py` surveille le bouton et pour chaque appui lance soit le démarrage du code `master.py` soit son arrêt propre. Ce code « maître » est celui qui gère tout le déroulement de la session. Il lance `gps.py`, `imu.py`

et `vid.py` qui gèrent respectivement les enregistrements du GPS, de l'IMU et de la caméra. À son extinction, il arrête ces trois processus et lance `grp.py`, dont le but est de finaliser l'enregistrement vidéo et de transformer les informations brutes en fichier `.mp4` lisible.

Tous ces fichiers s'appuient sur un fichier de classes commun, `classes.py`. Les logs s'enregistrent dans un fichier `sys.log`.

Pour la structure générale, j'ai choisi de séparer les différentes tâches en processus distincts et non pas en threads d'un même processus. Je ne pense pas qu'il y ait de « bonne » ou de « mauvaise » réponse à cette question, mais après avoir attentivement pesé le pour et le contre, j'ai choisi cette option en raison de l'excellente isolation qu'elle propose, ce qui est utile si un des processus venait à planter. Cependant, cette méthode complexifie grandement la communication entre les différentes tâches, les processus forçant l'utilisation de pipes.

Fichier de classes — `classes.py`

Beaucoup de fonctions et de code est commun à plusieurs de fichiers utilisés. Afin d'éviter de réécrire dans chaque fichiers ces parties, on a recours à un fichier de classes. Il suffit de l'importer dans chaque autre fichier avec `from classes import *`

Dans ce fichier, on règle notamment le logging. Ainsi, à chaque fois qu'un log est effectué, quel que soit le fichier, il sera formaté convenablement, puis imprimé dans la console avec la couleur appropriée et enregistré dans le fichier de log `sys.log`.

On définit également la classe `SysDataBase` qui servira à interagir avec la base de donnée `sys.db`. Elle permet d'utiliser `cursor` sans avoir à l'initialiser manuellement, d'accéder facilement à l'identifiant de la session en cours, et de finaliser proprement un enregistrement.

Finalement, on retrouve aussi la définition de la fonction décoratrice `non_blocking`. Elle prend en argument une fonction et en retourne la version non bloquante (tournant dans un thread séparé). Elle est utilisée pour modifier la fonction de reboot d'urgence, également définie dans ce fichier.

```
python
#!/usr/bin/env python
import sqlite3
import colorlog
import logging
import threading
import time
import subprocess

handler = colorlog.StreamHandler()
handler.setFormatter(colorlog.ColoredFormatter(
    "%(log_color)s%(asctime)s | %(filename)s | %(levelname)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S"
))
log = logging.getLogger()
log.addHandler(handler)

file_handler = logging.FileHandler('sys.log')
file_formatter = logging.Formatter(
    "%(asctime)s | %(filename)s | %(levelname)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S"
)
file_handler.setFormatter(file_formatter)
log.addHandler(file_handler)

log.setLevel(logging.DEBUG)

class SysDataBase:
    def __init__(self, filename):
        self.filename = filename

        self.db_name = "sys.db"
        # Connexion inter-thread pour atteindre les callbacks gpiozero (thread
distant)
        self.conn = sqlite3.connect(self.db_name, check_same_thread=False)
        self.cursor = self.conn.cursor()
        self.session_id = self.conn.execute("SELECT max(id) FROM
sessions;").fetchone()[0]
        self.session_id = int(self.session_id) if self.session_id else 0

    def terminate_database_connexion(self):
        try:
            self.conn.commit()
        except Exception as e:
            logging.warning(f"The final flush was interrupted: {e}")
        try:
            self.conn.close()
        except Exception as e:
            logging.error(f"Failed to close the database connection: {e}")
```



```
python
def actualise_session_id(self):
    self.session_id += 1

def non_blocking(func):
    def wrapper(*args, **kwargs):
        thread = threading.Thread(target=func, args=args, kwargs=kwargs)
        thread.start()
    return wrapper

@non_blocking
def emergency_reboot():
    logging.warning("Emergency reboot initiated")
    time.sleep(1)
    subprocess.Popen(["sudo", "reboot"])
```

Je n'ai pas créé ce fichier dès le départ, mais à mesure que je codais et que je voyais les répétitions s'accumuler, il est devenu évident qu'il était nécessaire de le créer.

Surveillance du bouton — interface.py

Le bouton est branché entre le pin 18 (BCM24) et la terre. Il n'est pas nécessaire d'ajouter de résistance car le GPIO 24 est réglé en pull up interne.

On commence par créer une instance de la classe Button qui sera utilisée pour la surveillance du pin. On définit l'événement `when_pressed` de cette instance comme un daemon. Il attend en arrière plan. Quand il se réveille, il appelle le callback `bouton_pressed` puis meurt immédiatement.

Lors de l'exécution de la fonction `bouton_pressed`, le flag global `witness_button` passe à `True` et le callback créé par le daemon meurt rapidement (aucune action bloquante ne peut être exécutée dedans).

L'inversion du flag est détectée par la boucle infinie et déclenche, selon le statut du booléen `session_is_running`, soit le lancement de `master.py`, soit l'envoi du signal `SIGUSR1` qui sera intercepté par `master.py` et provoquera son extinction. Le booléen `session_is_running` est ensuite inversé pour que l'action exécutée au prochain appui soit correct, un `time.sleep(1)` est lancé pour éviter les rebonds, puis le flag `witness_button` retourne à `False`.

Dans les lignes suivant la boucle, on a la gestion d'erreurs. Un reboot est provoqué en cas d'interruption inopinée du script, car le prototype n'est d'aucune utilité s'il ne s'exécute plus.

```
python
#!/usr/bin/env python
from classes import *
import logging
import traceback
from gpiozero import Button
import subprocess
import os
import signal
import time
import sys

button = Button(24, pull_up=True)
witness_button = False
session_is_running = False
session = None

def bouton_pressed():
    global witness_button
    witness_button = True
button.when_pressed = bouton_pressed

logging.info("Starting")
logging.info("Interface is standing by, ready for a new session")

while True:
    try:
        if witness_button:
            logging.debug("Button pressed")
            if session_is_running:
                session_is_running = False
                if session:
                    logging.debug("Sending SIGUSR1 to master.py")
                    os.kill(session.pid, signal.SIGUSR1)
                    logging.info("Interface is standing by, ready for a new session")
            else:
                session_is_running = True
                logging.debug("Launching master.py")
                session = subprocess.Popen(["python3", "master.py"])
                time.sleep(1)
                witness_button = False

            time.sleep(0.1)
        except KeyboardInterrupt:
            logging.info("Shutting down due to a KeyboardInterrupt")
            if session_is_running and session:
                logging.debug("Terminating ongoing session")
                session.terminate()
            sys.exit(0)
```

```
python
except Exception as e:
    logging.critical(f"Major failure : a critical error crashed
                    interface.py : {e}")
    logging.warning("Attempting to safely terminate the ongoing session
                    and reboot")
    logging.critical(traceback.format_exc())
try:
    if session:
        session.terminate()
        logging.debug("Terminating ongoing session")
except Exception as e:
    logging.error("Failed to stop the ongoing session")
logging.warning("Interface.py is exiting")
emergency_reboot()
sys.exit(1)
```

Dans une précédente version, tout le contenu de la boucle était contenu dans l'événement `bouton_pressed()`, mais le callback créé pour son exécution n'est pas fait pour des tâches bloquantes et se terminait avant la fin de l'exécution du programme contenu dedans. Il est résultait des comportements imprévisibles particulièrement complexes à déboguer car issus d'effets de bords, tel que des inversions de booléens sans raison apparente.

Gestion de la session — master.py

L'interception du signal `SIGUSR1` est gérée à la ligne 12. Elle déclenche le callback `handle_usr1` qui passe le flag global `ending_session` à `True`. Ainsi, le callback reste court et non bloquant, ce qui est généralement une bonne pratique.

Ce script va dans un premier temps créer une instance la classe `SysDataBase` afin de pouvoir interagir avec la base de donnée (notamment récupérer l'identifiant de la dernière session). La première action concrète est de créer la nouvelle session. Son identifiant est déterminé automatiquement avec `AUTOINCREMENT` directement en SQL. la fonction de classe `actualise_session_id()` ajoute partiellement 1 au compteur afin de passer de l'identifiant de la session précédente à celui de celle qui vient d'être créée.

La création de la session est directement suivie du lancement des scripts `imu.py`, `gps.py` et `vid.py` qui liront et enregistreront les données des capteurs. On passe en argument l'identifiant de la session à `vid.py`, car mettant un peu plus de temps à démarrer, il est possible que le programme récupère le mauvais identifiant si les sessions sont très rapprochées. Lui donner le même identifiant de session que le master évite toute confusion.

Une fois les processus lancés, on attend de recevoir un signal avec `signal.pause` avant de continuer. Afin d'éviter de terminer le master en recevant un autre signal que celui qu'envoie `interface.py`, on ne continue que si le flag global `ending_session` est vrai (c'est-à-

dire si le callback `handle_usr1` a été déclenché par la réception de `SIGUSR1`, le signal envoyé par `interface.py`).

Pour l'extinction des sous-processus, on utilise la fonction `shutdown()`. Elle envoie à chacun un `SIGTERM` pour les terminer. Si ce n'est pas suffisant et qu'après 5 seconde ils sont encore vivants, elle les tue avec `SIGKILL`.

On termine ensuite l'enregistrement de la session en complétant le tableau SQL avec l'heure de fin, puis on la ferme en utilisant la fonction de classe prévue à cet effet.

Finalement, on lance `grp.py` afin qu'il finalise l'enregistrement vidéo correctement. Son exécution peut être lente, mais il tourne en arrière plan et un nouveau master peut directement être lancé par `interface.py` pour démarrer une nouvelle session.

```
python
#!/usr/bin/env python
from classes import *
import subprocess
import sys
import time
import logging
from datetime import datetime
import signal

ending_session = False

def handle_usr1(signum, frame):
    global ending_session
    ending_session = True
    signal.signal(signal.SIGUSR1, handle_usr1)

def shutdown():
    for proc in processes:
        if proc.poll() is None:
            proc.terminate()
    time.sleep(5)
    for proc in processes:
        if proc.poll() is None:
            logging.warning(f"Process {proc} did not terminate in time, sending SIGKILL.")
            proc.kill()

sys_data_base = SysDataBase(__file__)
logging.info("Starting")

try:
    timestamp = datetime.now().isoformat()
    sys_data_base.cursor.execute(
        "INSERT INTO sessions (start, stop, name) VALUES (?, ?, ?)",
        (timestamp, None, None)
    )
    sys_data_base.conn.commit()
    sys_data_base.actualise_session_id()
    logging.debug(f"Session {sys_data_base.session_id} created")
except Exception as e:
    logging.error(f"Failed to create a new session : {e}")
    sys.exit(1)

try:
    logging.debug("Launching imu.py")
    logging.debug("Launching gps.py")
    logging.debug("Launching vid.py")
```

```
python
    processes = [
        subprocess.Popen(["python3", "imu.py"]),
        subprocess.Popen(["python3", "gps.py"]),
        subprocess.Popen(["python3", "vid.py", str(sys_data_base.session_id)]),
    ]
except Exception as e:
    logging.critical(f"Failed to launch the dependencies : {e}")
    shutdown()
    sys.exit(1)

try:
    while not ending_session:
        signal.pause()
except Exception as e:
    logging.critical(f"Failed to terminate the subprocesses : {e}")
    shutdown()
    sys.exit(1)

if ending_session:
    shutdown()
    for proc in processes:
        proc.wait()
    logging.debug("All subprocesses terminated.")

    try:
        timestamp = datetime.now().isoformat()
        sys_data_base.cursor.execute(
            "UPDATE sessions SET stop = ? WHERE id = (SELECT max(id) FROM sessions);",
            (timestamp,)
        )
        sys_data_base.terminate_database_connexion()
        logging.debug(f"Session {sys_data_base.session_id} ended, connexion closed")
    except Exception as e:
        logging.error(f"Failed to properly end the session : {e}")
        sys.exit(1)

    logging.debug("Session properly ended")
    logging.debug("launching grp.py")
    fusion = subprocess.run(["python3", "grp.py", str(sys_data_base.session_id)])
    logging.info(f"Master terminated for session {sys_data_base.session_id}")
    sys.exit(0)
```

De même que pour `interface.py`, ce code n'a été nécessaire que pour la coordination des autres, et a été pour cette raison un des derniers à être écrit. Aucune syntaxe de ce code n'est particulièrement complexe, mais il représente la clef de voûte de la logique du code et de son architecture. Il a à ce titre nécessité une importante réflexion préalable.

Gestion de l'IMU et du GPS — imu.py & gps.py

Ces deux codes, bien que distincts, fonctionnent exactement de la même manière. Ils sont tous deux basés sur le code fourni par le fabricant et présenté dans le Chapitre 11 et le Chapitre 12.

Tout d'abord, on trouve la gestion des signaux d'arrêts. Quand on reçoit un SIGTERM ou un SIGINT, le callback `stop_handler` est appelé pour passer le flag global `running` à `False`, ce qui interrompra proprement la boucle de lecture et d'enregistrement des données.

Suit la fonction `acquisition()` contenant le code du fabricant pour la récupération des données. On définit ensuite une instance de classe `SysDataBase` pour l'enregistrement des données, la fréquence et la période, ainsi que d'autres variables comme dans le code du fabricant.

Dans la boucle, on commence par acquérir les données avec la fonction prévue à cet effet, puis on réalise un enregistrement dans la base de donnée. On attend ensuite le temps restant avant le prochain enregistrement (fréquence - temps passé pour cet enregistrement).

L'écriture n'est validée qu'une fois par seconde afin de ne pas surcharger le Raspberry avec des opérations inutiles. Pour ce faire, on utilise un compteur. Lorsqu'il atteint la fréquence, alors une seconde s'est écoulée, l'enregistrement est réalisé et le compteur réinitialisé.

Finalement, on essaye de valider les derniers enregistrements puis on ferme la connexion. Ces deux opérations sont effectuées par la fonction de classe `terminate_database_connexion()`.

imu.py


```
python
#!/usr/bin/env python
from classes import *
import time
import signal
import sqlite3
from datetime import datetime
from icm20948 import ICM20948

# Interruption depuis master.py ou par Ctrl+C
running = True
def stop_handler(sig, frame):
    global running
    running = False
signal.signal(signal.SIGTERM, stop_handler)
signal.signal(signal.SIGINT, stop_handler)

def acquisition():
    mag_x, mag_y, mag_z = imu.read_magnetometer_data()
    accel_x, accel_y, accel_z, gyro_x, gyro_y, gyro_z =
    imu.read_accelerometer_gyro_data()
    return mag_x, mag_y, mag_z, accel_x, accel_y, accel_z, gyro_x, gyro_y, gyro_z

sys_data_base = SysDataBase(__file__)

imu = ICM20948()
Hz = 5
period = 1 / Hz

logging.info(f"Starting")
logging.debug(f"Saves the imu informations into {sys_data_base.db_name}")

compteur = 0
try:
    while running:
        start = time.monotonic()
        compteur += 1
        timestamp = datetime.now().isoformat()

        imu_data = acquisition()

        try:
            sys_data_base.cursor.execute(
                "INSERT INTO imu_data (timestamp, accel_x, accel_y, accel_z,
                                         gyro_x, gyro_y, gyro_z,
                                         mag_x, mag_y, mag_z, session_id)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
                (timestamp, *imu_data, sys_data_base.session_id)
            )

```

```
python
    # Valider l'écriture une fois par seconde
    if compteur >= Hz:
        compteur = 0
        sys_data_base.conn.commit()

    except sqlite3.IntegrityError as e:
        logging.error(f"Integrity error: {e}")

    # Gestion de la fréquence
    remaining = period - (time.monotonic() - start)
    if remaining > 0 and running:
        time.sleep(remaining)

except Exception as e:
    logging.error(f"The execution was interrupted: {e}")

finally:
    sys_data_base.terminate_database_connexion()
    logging.info(f"__file__ terminated")
```

gps.py

```
python
#!/usr/bin/env python
from classes import *
import time
import signal
from datetime import datetime
import board
import busio
import adafruit_gps
import serial
import sqlite3

# Interruption par master.py ou par Ctrl+C
running = True
def stop_handler(sig, frame):
    global running
    running = False
signal.signal(signal.SIGTERM, stop_handler)
signal.signal(signal.SIGINT, stop_handler)

def acquisition():
    # Obligatoires
    lat = f"{gps.latitude:.6f}"
    lon = f"{gps.longitude:.6f}"
    p_lat = f"{gps.latitude_degrees} degs, {gps.latitude_minutes:2.4f} mins"
    p_lon = f"{gps.longitude_degrees} degs, {gps.longitude_minutes:2.4f} mins"
    fix_qual = gps.fix_quality

    # Facultatifs
    n_satellites = gps.satellites
    alt = gps.altitude_m
    alt_geoid = gps.height_geoid
    sog_kn = gps.speed_knots
    sog_kmh = gps.speed_kmh
    cog = gps.track_angle_deg
    dilution = gps.horizontal_dilution

    return lat, lon, p_lat, p_lon, fix_qual, n_satellites, alt, alt_geoid, sog_kn,
    sog_kmh, cog, dilution

sys_data_base = SysDataBase(__file__)

rx = board.RX
tx = board.TX
uart = serial.Serial("/dev/serial0", baudrate=9600, timeout=10)

# Create an GPS instance
gps = adafruit_gps.GPS(uart, debug=False)
```

```
python
# Turn on the basic GGA and RMC info, then set update rate to 2Hz
gps.send_command(b"PMTK314,0,1,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0")
Hz = 2
period = 1/Hz
gps.send_command(b'PMTK220,(period*1000)')

logging.info(f"Starting")
logging.debug(f"Saves the gps informations into {sys_data_base.db_name}")

try:
    c = 0
    while running:
        gps.update()
        if not gps.has_fix:
            logging.warning(f"GPS does not have a fix, waiting for it")
            time.sleep(1)
            continue

        # If the GPS has a fix
        timestamp = datetime.now().isoformat()
        start = time.monotonic()
        c += 1

        gps_data = acquisition()

        # Requête SQL
        try:
            sys_data_base.cursor.execute(
                "INSERT INTO gps_data (timestamp, lat, lon, p_lat, p_lon,
fix_qual, n_satellites, alt, alt_geoid, sog_kn, sog_kmh, cog, dilution, session_id)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
                (timestamp, *gps_data, sys_data_base.session_id)
            )
            # Valider l'écriture
            if c >= Hz:
                c = 0
                sys_data_base.conn.commit()

        except sqlite3.IntegrityError as e:
            logging.warning(f"Integrity error: {e}")

        # Gestion de la fréquence
        remaining = period - (time.monotonic() - start)
        if remaining > 0 and running:
            time.sleep(remaining)
```

```
python
except Exception as e:
    logging.error(f"The execution was interrupted: {e}")

finally:
    sys_data_base.terminate_database_connexion()
    logging.info(f"{'__file__'} terminated")
```

Ces deux codes sont issus du remaniement d'autres codes. Ils sont les premiers que j'ai écrit pour mon TM et ont beaucoup évolué. À partir d'une simple lecture de données imprimées dans la console, ils sont devenus un système d'enregistrement entreliant python et requêtes SQL.

Bien que moins complexes que certains autres codes, ils m'ont servi de laboratoires pour des syntaxes qui m'étaient inconnues, tel que le logging, les requêtes SQL, les callbacks, les signaux et la structure try/except/finally. Je n'avais jamais fait de gestion d'exception auparavant, et travailler avec des capteurs physiques est exigeant sur ce plan-là, puisque les entrées ne sont par définitions par contrôlées.

Enregistrement de la vidéo – vid.py

On commence par récupérer l'identifiant de la session qui a été passé avec `subprocess.Popen`. Si on lançait l'enregistrement indépendamment du master, l'identifiant serait récupéré à partir de la base de donnée.

On crée les dossiers dans lesquels seront enregistrés les vidéos. Le dossier de destination est nommé d'après l'identifiant de la session, il se trouve dans le dossier sessions.

Avant de commencer à enregistrer, on crée un thread dont le rôle est de surveiller la création des premiers segments vidéos. Ainsi, on a le timestamp le plus précis possible quand au début de la vidéo. Il peut persister un léger décalage dû au fait que l'on ne surveille pas le début de la prise de vue mais celui de l'enregistrement. Malheureusement, il n'est pas possible d'obtenir d'indication plus précise quand au timestamp du démarrage de la vidéo. On définit le thread comme un daemon afin qu'il ne survive pas après la mort de son programme maître.

On exécute ensuite avec `subprocess.Popen` la commande bash lançant l'enregistrement de la vidéo. Cette commande est tirée de la documentation de `rpicalm-vid`⁴. On enregistre la vidéo par segment de deux seconde en H264 brut. Ainsi, le risque d'obtenir une vidéo endommagée est au plus faible. En effet, des formats complexes tels que MP4 nécessitent un encodage final, alors que H264 reste lisible même en cas d'interruption brutale de l'enregistrement sans finalisation appropriée.

⁴Documentation disponible sur le [site officiel de Raspberry Pi](#).

Le code attend ensuite de recevoir le signal d'extinction. Une fois reçu, il essaye de terminer l'enregistrement de la vidéo en envoyant SIGTERM. Si au bout de quatre secondes le processus, il envoie SIGKILL pour le tuer.

Si une connexion à la base de données avait été ouverte, elle est finalement fermée.

```
python
#!/usr/bin/env python
from classes import *
import subprocess
import sys
import signal
import threading
import time
import os
from datetime import datetime

# Interruption depuis master.py ou par Ctrl+C
running = True
def stop_handler(sig, frame):
    global running
    running = False
signal.signal(signal.SIGTERM, stop_handler)
signal.signal(signal.SIGINT, stop_handler)

sys_data_base = False
try:
    session_id = sys.argv[1]
except Exception as e:
    sys_data_base = SysDataBase(__file__)
    session_id = sys_data_base.session_id

start_time = None
first_folder = 'sessions'
os.makedirs(first_folder, exist_ok=True)
folder = os.path.join(first_folder, str(session_id))
os.makedirs(folder, exist_ok=True)

logging.info(f"Starting")
logging.debug(f"Destination folder for the .h264 videos : {folder}")

def watch_first_segment(folder):
    global start_time
    first_file = os.path.join(folder, "video_00000.h264")

    while not os.path.exists(first_file):
        time.sleep(0.01)

    start_time = datetime.now()
    with open(os.path.join(folder, "start.txt"), "w", encoding="utf-8") as fichier:
        fichier.write(str(start_time))

watcher = threading.Thread(target=watch_first_segment, args=(folder,))
watcher.daemon = True
watcher.start()
```



```
python
# Interruption externe
running = True
def stop_handler(sig, frame):
    global running
    running = False
signal.signal(signal.SIGTERM, stop_handler)
signal.signal(signal.SIGINT, stop_handler)

cmd = [
    "rpicas-vid",
    "-t", "0",
    "--segment", "2000",
    "--inline",
    "-o", os.path.join(folder, "video_%05d.h264"),
    "--nopreview",
    "--codec", "h264"
]

try:
    proc = subprocess.Popen(cmd)
except Exception as e:
    logging.error(f"Error at the launching: {e}")
    sys.exit(1)

try:
    while running:
        signal.pause()

finally:
    if proc.poll() is None:
        proc.terminate()
        try:
            proc.wait(timeout=4)
        except subprocess.TimeoutExpired:
            logging.warning("Process did not terminate_database_connexion in time,
sending SIGKILL.")
            proc.kill()
    if sys_data_base:
        logging.debug("Closing database connection")
        sys_data_base.conn.close()
    logging.info(f"{{__file__}} terminated")
```

Le code pour l'enregistrement de la vidéo a sans doute été celui qui a été le plus complexe pour moi à écrire sur le plan technique.

En effet, il était pour moi désorientant de ne pas pouvoir lancer une vidéo sans indiquer de fin prévue. J'ai mis du temps à comprendre qu'il fallait en réalité en lancer plusieurs courtes d'une durée définie pour en composer une grande.

De plus, j'essayais au début d'enregistrer au format MP4, mais les fichiers que j'obtenais étaient systématiquement corrompus. Même en passant sur du H264 brut, j'avais encore le problème que la première image de chaque séquences ne coïncidaient pas la dernière de la séquence précédente. Ce problème a été résolu par l'ajout de `--inline` qui jalonne clairement le début et la fin de chaque séquence.

Finalement, j'ai eu beaucoup de difficulté à obtenir un timestamp précis pour le début de la vidéo, étant donné que `rpicas-vid` n'en fournit pas de lui-même et qu'il y a quasiment deux secondes de décalage entre l'exécution de la commande et le début réel de la vidéo. La solution que j'ai adoptée n'est pas parfaite, mais je n'en ai pas de meilleur pour l'instant.

Finalisation de l'enregistrement de la vidéo — `grp.py`

De même que pour `vid.py`, on essaye de récupérer l'identifiant de la session via un passage d'argument système depuis `master.py`. Si on y parvient pas, on utilise l'identifiant de la session la plus récente présente dans la base de données. On définit ensuite les chemins absolu et relatif du dossier dans lequel sont enregistrées les vidéos H264 à assembler.

On crée un fichier texte contenant la liste des chemins absolus de chaque extrait H264 classée par ordre chronologique pour FFmpeg. On récupère également le timestamp du début d'enregistrement de la vidéo.

Une fois cela fait, on peut exécuter la commande utilisant FFmpeg pour la fusion des vidéos indiquées dans le fichier texte. Pour gagner du temps processeur, on ne ré-encode pas chaque vidéo, on se contente de les concaténer puis d'enrober le résultat dans un MP4. On passe en métadonnée l'heure de création. La vidéo finale s'appelle `final.mp4` et est enregistrée à côté des extraits dont elle provient.

Finalement, on supprime le fichier texte ayant servi à FFmpeg, puis on ferme la connexion à la base de donnée si elle a été ouverte.

```
python
#!/usr/bin/env python
from classes import *
import subprocess
import os
import sys

sys_data_base = False
try:
    session_id = sys.argv[1]
except Exception as e:
    sys_data_base = SysDataBase(__file__)
    session_id = sys_data_base.session_id
relative_path = os.path.join('sessions', str(session_id))
absolute_path = os.path.join(os.getcwd(), relative_path)

logging.info(f"Starting")
logging.debug(f"Saving the final .mp4 video as {relative_path}/final.mp4")

try:
    files = []
    for video in os.listdir(relative_path):
        if video.endswith(".h264"):
            files.append(os.path.join(absolute_path, video))
    files.sort()

    with open(os.path.join(relative_path, 'start.txt'), 'r', encoding='utf-8')
as f:
    timestamp = f.read()

    with open("list.txt", "w") as f:
        for file in files:
            f.write(f"file '{os.path.abspath(file)}'\n")

    command = [
        'ffmpeg',
        '-fflags', '+genpts',
        '-f', 'concat',
        '-safe', '0',
        '-i', 'list.txt',
        '-c', 'copy',
        '-metadata', f'creation_time={timestamp}',
        os.path.join(relative_path, f"final.mp4")
    ]
    try:
        subprocess.run(command)
    except Exception as e:
        logging.error(f"FFmpeg raised an error : {e}")
        sys.exit(1)
```

```
python
os.remove("list.txt")
    logging.debug("Videos properly merged")

except Exception as e:
    logging.error(f"Failed to merge videos : {e}")
    sys.exit(1)

finally:
    if sys_data_base:
        logging.debug("Closing database connection")
        sys_data_base.conn.close()
    if os.path.exists("list.txt"):
        os.remove("list.txt")
    logging.info(f"{{__file__}} terminated")
```

Une fois le fichier `vid.py` créé, la majorité des problèmes concernant la vidéos avaient déjà été tirés au clair. L'écriture de ce fichier n'a pas été particulièrement complexe étant donné qu'elle découle de la résolution des problèmes qui ont été rencontrés. La seule surprise a été de constater qu'il n'était pas possible de fusionner les vidéos via python directement, mais qu'il fallait passer par une commande `bash`.

16 CRÉATION DES SCHÉMAS

Le schéma de la base de donnée a été réalisé avec Miro, un logiciel permettant notamment de créer des visualisation en ligne rapidement et gratuitement.

Pour la créations des schémas concernant les câblages, j'ai utilisé le logiciel Fritzing. Bien que j'accorde une grande importance à utiliser des logiciels open source, j'ai choisi celui-ci en raison de la rapidité avec laquelle il permet de créer des dessins propres, mais surtout en raison de la large disponibilité de composants graphiques mis à dispositions, eux, en open source, par les utilisateurs.

En effet, il existe un grand nombre de composants informatique, et tous ne peuvent pas être mis à disposition par Frizing. Pour cette raison, certains utilisateurs (ou directement les fabricants) mettent à disposition des modèles de certains breakout pour pouvoir les utiliser dans des schémas. Voici la liste de ceux que j'ai téléchargé au format `.fzpz` :

- [Adafruit PowerBoost 1000C](#)
- [Adafruit Ultimate GPS Breakout V3](#)
- [ICM20948](#) (le breakout de Pimoroni n'étant pas disponible, j'ai choisi d'utiliser celui d'Adafruit. Ils ont le même pinout.)